Trace Volume and Sampling, Worked Out by Hand

Why one agent trajectory is eleven spans, not one log line —
and exactly how hard you can sample before p99 goes blind.

What this document does. It takes the one production trajectory from Module 4.2 — the *refund_agent@1.4.2* run whose trace tree shows a `graph_run` root over `plan` / `act` nodes, `llm_call`s and `tool_call`s — and counts the *spans* it emits. From one round number per span we then compute the storage bill at the module's fleet rate (the dashboard quotes *12,430 trajectories/hr*), and finally derive how far you can turn down a head-sampling rate r before the `p99` cost/latency tiles stop being trustworthy. Nothing is hand-waved: every figure below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent.

Where this sits. This is **Topic 1 of Module 4.2** (Agent Observability, Track 4) — the quantitative floor under the module's claim that "a logged line per request is enough for RAG; agents need trajectory-aware traces." Its companions: Module 0.0 (the ReAct cost loop — the *same* step count N, now driving span volume instead of dollars) and Module 3.4 (cost engineering, which alerts on the very `p99` this document teaches you not to sample away).

Concepts covered, in order: a trajectory as a span tree · spans-per-trace $S = 1 + N(1 + t)$ · volume $= R \cdot S \cdot$ seconds · storage = spans × bytes × retention · the 11× blow-up vs a single-span RAG log · head sampling at rate r: storage scales by r · quantile standard error $\sim 1/\sqrt{r\,N}$ · how low r can go while p99 stays stable · tail-based sampling — trace everything in incidents, sample heavily in steady state.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one trajectory, counted span by span

In RAG observability, one user request produces *one* trace with essentially one span — the LLM call. An agent is different. Module 4.2's trace tree for the refund run is a *tree*: a root `graph_run` span, then for each agent step a `graph_node` that contains one `llm_call` (the model decides) plus the `tool_call` spans it fires. We will fold the node into its LLM span and count, per step, **one LLM span + t tool spans**, with a single root span over the whole trajectory. That is the only structural fact we need; the rest is counting.

The quantities. Every number is round so the sums follow on paper. The mechanics are identical at any size — only the constants move.

Quantity	Symbol	Value here
Agent steps in the trajectory	N	5
Tool calls per step	t	1
Spans per trajectory	$S = 1 + N(1+t)$	derived: 11
Incoming traffic (traces/second)	R	50 / s
Bytes per span (attributes + payload)	b	2,048 B (2 KB)
Retention window	D	7 days
Seconds per day	—	86,400

Notation. S is spans per trace. N_{total} is the number of *traces* kept over the retention window. A *head* sampling rate $r \in (0, 1]$ is decided at trace start: keep a fraction r of traces, drop the rest, before any spans are written.

Intuition

A RAG log is a receipt: one line, one request. An agent trace is a *flight recorder*: the whole trajectory — every node it entered, every model call, every tool it fired, every retry. You cannot reconstruct a five-step trajectory from one span any more than you can reconstruct a flight from a single altitude reading. That is the entire reason agents need trace volume that a chatbot never did: the unit of truth is the *path*, and a path has many points.

2 Step 1 — how many spans one trajectory emits

Walk the tree top-down. There is exactly one root span for the trajectory. Then each of the N steps contributes one LLM span (the model's decision) and t tool spans (the tools it called). So

$$S = \underbrace{1}_{\text{root}} + N \underbrace{(1+t)}_{\text{per step}}. \tag{1}$$

For the refund trajectory, $N = 5$ and $t = 1$, so $S = 1 + 5 \cdot (1 + 1) = 1 + 10 = 11$ spans.

Mini-example — this operation in isolation

Count the module's own tree by hand and you land in the same place: a `graph_run` root, then `plan` (1 LLM call), `act` iteration 1 (LLM + tools), `act` iteration 2 (LLM + tools, plus a retry), and `END`. Collapsing each node onto its LLM call and charging one span per tool, a clean five-step, one-tool trajectory is $1 + 5 \cdot 2 = 11$ spans. A RAG request over the *same* traffic is $S = 1$. The agent emits $11\times$ the spans of the chatbot for a single user question — and that multiplier is the whole story of why the storage bill explodes.

3 Step 2 — from spans to a storage bill

Now attach bytes and time. Traffic of R traces per second, sustained, is $R \cdot 86{,}400$ traces per day. Each trace is S spans; each span is b bytes; we keep them for D days. The retained volume is a plain product:

$$\text{spans/day} = R \cdot 86{,}400 \cdot S, \qquad \text{bytes} = R \cdot 86{,}400 \cdot S \cdot b \cdot D.$$

Working the numbers at $R = 50$, $S = 11$, $b = 2{,}048$, $D = 7$:

Quantity	**Value**	**how**
Traces / day	4,320,000	$50 \times 86{,}400$
Spans / day	47,520,000	$\times 11$
Spans retained (7d)	332,640,000	$\times 7$
Bytes retained	681,246,720,000	$\times 2{,}048$
Storage	**681.25 GB**	$/10^9$ (decimal GB)

Sample check: $4{,}320{,}000 \times 11 = 47{,}520{,}000$ spans/day; over 7 days that is 332,640,000 spans; at 2,048 B each, 681,246,720,000 B = 681.25 GB. ✓ (In binary units that is 634.46 GiB — same bytes, different divisor.)

Now the RAG comparison. Run the *identical* traffic through a single-span log: $S = 1$ instead of 11. Everything else cancels, so the storage is exactly $681.25/11 = 61.93$ GB. The agent's flight recorder costs 11× the RAG receipt — the blow-up factor *is* S.

Watch out

The 50 / s here is illustrative, but the module's dashboard quotes *12,430 trajectories/hr* ≈ 3.5 / s for one agent. A real fleet runs many agents, many tenants, and chattier tool payloads (a verbose observation can push b well past 2 KB). The point is not the exact 681 GB — it is that the bill scales *linearly in* S, and S is the one number RAG intuition gets wrong by an order of magnitude. Budget for spans-per-trace, not requests-per-second.

4 Step 3 — head sampling: the lever and its cost

You cannot afford to keep every trajectory forever, and you should not have to. *Head* sampling decides at trace start to keep a fraction r of traces. Because the decision is per-trace and uniform, storage scales *exactly* by r:

$$\boxed{\text{Storage}(r) \;=\; r \cdot \big(R \cdot 86{,}400 \cdot S \cdot b \cdot D\big)} \tag{2}$$

What does it cost you? The dashboards in Module 4.2 live on *percentiles* — p50/p95/p99 cost and latency. A percentile estimated from n sampled traces has a sampling error that shrinks like

$$\text{SE}\big(\widehat{q}\big) \;\sim\; \frac{1}{\sqrt{r\, N_{\text{total}}}}, \tag{3}$$

the usual $1/\sqrt{n}$ law with $n = r\, N_{\text{total}}$ kept traces. Halving r does *not* halve your accuracy — error grows as the inverse square root, so it takes a 100× cut in r to make the estimate 10× noisier. That asymmetry is the whole reason sampling works.

Mini-example — this operation in isolation

The $\sqrt{}$ is doing the heavy lifting. Over the 7-day window at 50 / s, $N_{\text{total}} = 50 \cdot 86{,}400 \cdot 7 = 30{,}240{,}000$ traces. At $r = 1.0$ the relative standard error is $1/\sqrt{30.24\text{M}}$; at $r = 0.01$ it is $1/\sqrt{0.3024\text{M}}$ — a ratio of exactly $\sqrt{1/0.01} = \sqrt{100} = 10\times$. You threw away 99% of your storage and made p99 only ten times noisier — and as the next step shows, ten-times-noisier

here is still plenty sharp.

5 Step 4 — how low can r go before p99 goes blind?

p99 is, by definition, the top 1% of traces. To estimate it stably you need a healthy *count* of traces in that tail. If you keep $r\,N_{\text{total}}$ traces, the p99 tail holds about $r\,N_{\text{total}} \cdot 0.01$ of them. A common rule of thumb: keep at least a *few hundred* tail samples so the estimate does not jump around between refreshes. Setting that floor at 300:

$$r\,N_{\text{total}} \cdot 0.01 \;\geq\; 300 \quad\Longrightarrow\quad r \;\geq\; \frac{300}{0.01\,N_{\text{total}}} \;=\; \frac{300}{0.01 \cdot 30{,}240{,}000} \;=\; 0.00099.$$

So at this volume you could in principle sample as low as $r \approx 0.001$ and still keep $\sim$300 traces in the p99 tail. The whole landscape:

rate r	kept traces	storage	p99-tail traces	rel. SE
1.0	30,240,000	681.25 GB	302,400	1.0×
0.1	3,024,000	68.12 GB	30,240	3.2×
0.01	302,400	6.81 GB	3,024	10.0×

Read the table as a dial. At $r = 0.01$ you have cut storage 100-fold to 6.81 GB, yet you still retain 3,024 traces in the p99 tail — ten times the 300-trace floor — and your percentile estimate is only 10× noisier than the full-fidelity number. Sample check, $r = 0.01$: $30{,}240{,}000 \times 0.01 = 302{,}400$ kept; $\times 0.01 = 3{,}024$ tail traces; $681.25 \times 0.01 = 6.81$ GB; relative SE $= \sqrt{1/0.01} = 10$. ✓

Intuition

Sampling is not "throwing data away" — it is spending your storage budget where it buys information. The first thousand traces tell you almost everything about the *shape* of your latency distribution; the millionth tells you almost nothing new. The $1/\sqrt{n}$ curve is flat far out, which is exactly why $r = 0.01$ can be nearly as good as $r = 1.0$ for steady-state percentiles — while costing 1% as much to keep.

6 Step 5 — the asymmetry: steady state vs. incidents

Two regimes, one rule. In **steady state**, percentiles are all you need and the $1/\sqrt{n}$ law says you can sample hard — $r = 0.01$ keeps the dashboards honest at 1% of the cost. But during an **incident** you are not estimating a percentile; you are trying to read *one specific broken trajectory* end to end. For that, $r = 0.01$ is a disaster: there is a 99% chance the exact trace you need was never written.

Watch out

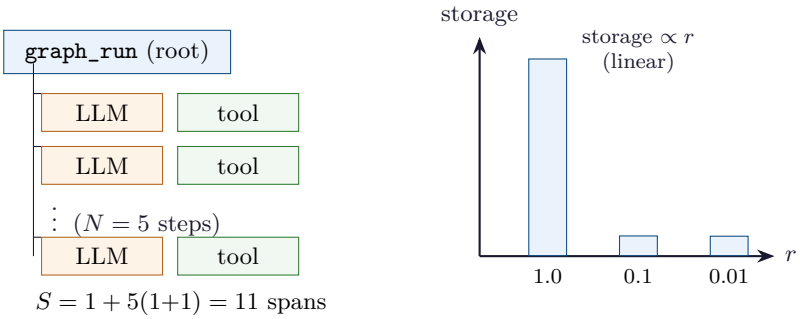
A sampled-away trajectory is gone forever — head sampling drops it *before* any span is recorded, so no amount of querying brings it back. This is why mature stacks pair a low steady-state r with **tail-based** (a.k.a. dynamic) sampling: keep 100% of traces that error, retry, hit the step cap, or exceed a latency/cost threshold, and sample the boring successes.

You decide *after* the trajectory finishes, when you know whether it was interesting. The module's "Trace volume drop" alert exists precisely because a mis-tuned sampler silently blinds you.

Intuition

The mental model: *trace EVERYTHING during incidents, sample HEAVILY in steady state.* Percentiles survive aggressive sampling because they are statistics over many traces; a debugging session does not, because it needs the *one* trace. You cannot debug a trajectory from one span — and you cannot debug it at all if the sampler already deleted it. Tune r for the dashboards; tune your tail rules for the pager.

7 The accounting, drawn



Left: one trajectory is a root span over N steps, each an LLM span plus its tools — eleven spans, not one. Right: head sampling at rate r scales that whole pile linearly. The picture *is* Equations (1) and (2).

8 Trace-volume cheat-sheet

Spans per trajectory	$S = 1 + N(1 + t)$
Traces per day	$R \cdot 86{,}400$
Spans retained	$R \cdot 86{,}400 \cdot S \cdot D$
Storage (bytes)	$R \cdot 86{,}400 \cdot S \cdot b \cdot D$
Storage under sampling	$\text{Storage}(r) = r \cdot (\text{full storage})$
RAG blow-up factor	S (agent spans $\div$ 1 RAG span)
Quantile standard error	$\text{SE} \sim 1/\sqrt{r\,N_{\text{total}}}$
p99-tail trace count	$r\,N_{\text{total}} \cdot 0.01 \geq$ a few hundred
Min head rate for p99	$r \geq 300/(0.01\,N_{\text{total}})$

9 An honest word about these numbers

The constants here ($N = 5$, $t = 1$, $b = 2\,\text{KB}$, $R = 50\,/\,\text{s}$, $D = 7\,\text{d}$) are illustrative round numbers, chosen so the storage product is followable on paper. Your real numbers will differ: a deeper agent raises N, a multi-tool step raises t, verbose tool outputs push b past 2 KB, and a busy fleet runs far above $50\,/\,\text{s}$. What is *not* illustrative is the structure. Spans-per-trace is $1 + N(1 + t)$, full stop; storage is a linear product of five factors; head sampling scales it by r exactly; and quantile error falls as $1/\sqrt{r\,N}$ — the inverse-square-root law that lets a 1% sample keep p99 honest. Change the constants and the GB figures move; the shapes do not. And one fact no tuning escapes: a trajectory dropped by the sampler is unrecoverable, which is why steady-state sampling and incident-time full-fidelity capture are two different dials, not one.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the span count, the storage bill, and the sampling table; change `N`, `t`, `R`, `BYTES`, or `RET_DAYS` to model your own fleet.

```python
# trace_volume.py -- reproduces every number in "Trace Volume and Sampling".
import math

def spans_per_trace(N, t): # S = 1 (root) + N*(1 LLM + t tools)
    return 1 + N * (1 + t)

N, t = 5, 1
S = spans_per_trace(N, t)
print(f"spans/trace N={N} t={t}: 1 + {N}*(1+{t}) = {S}")

R = 50 # incoming traces per second
BYTES = 2048 # bytes per span (2 KB)
RET_DAYS = 7 # retention window
SEC_DAY = 86400

traces_day = R * SEC_DAY
spans_day = traces_day * S
spans_kept = spans_day * RET_DAYS
bytes_kept = spans_kept * BYTES
gb = bytes_kept / (1000**3) # decimal GB

print(f"traces/day = {traces_day:,}")
print(f"spans/day = {spans_day:,}")
print(f"spans retained 7d = {spans_kept:,}")
print(f"bytes retained = {bytes_kept:,}")
print(f"storage GB = {gb:.2f}")

rag_gb = (traces_day * 1 * RET_DAYS * BYTES) / (1000**3) # S=1 baseline
print(f"RAG 1-span GB = {rag_gb:.2f} -> blow-up = {gb/rag_gb:.1f}x")

# --- head sampling table ---
N_total = traces_day * RET_DAYS # traces kept over the window
base_se = 1.0 / math.sqrt(N_total)
print(f"N_total traces 7d = {N_total:,}")
for r in (1.0, 0.1, 0.01):
    kept = N_total * r
    p99 = kept * 0.01 # traces in the top 1% tail
    se_rel = (1.0 / math.sqrt(kept)) / base_se
    print(f"r={r:<4} kept={kept:12,.0f} GB={gb*r:7.2f} "
          f"p99_tail={p99:11,.0f} relSE={se_rel:4.1f}x")

```

```python
r_min = 300 / (0.01 * N_total) # floor: >=300 tail traces
print(f"r_min for >=300 p99-tail traces = {r_min:.5f}")

# Expected output:
# spans/trace N=5 t=1: 1 + 5*(1+1) = 11
# traces/day = 4,320,000
# spans/day = 47,520,000
# spans retained 7d = 332,640,000
# bytes retained = 681,246,720,000
# storage GB = 681.25
# RAG 1-span GB = 61.93 -> blow-up = 11.0x
# N_total traces 7d = 30,240,000
# r=1.0 kept= 30,240,000 GB= 681.25 p99_tail= 302,400 relSE= 1.0x
# r=0.1 kept= 3,024,000 GB= 68.12 p99_tail= 30,240 relSE= 3.2x
# r=0.01 kept= 302,400 GB= 6.81 p99_tail= 3,024 relSE=10.0x
# r_min for >=300 p99-tail traces = 0.00099
```

— end of document —